
A Unified Identity Platform for Multi-Product SaaS

Design, Implementation, and Evaluation

Kalyan Inturi

Hampton Roads SQL Server User Group • Feb 2026

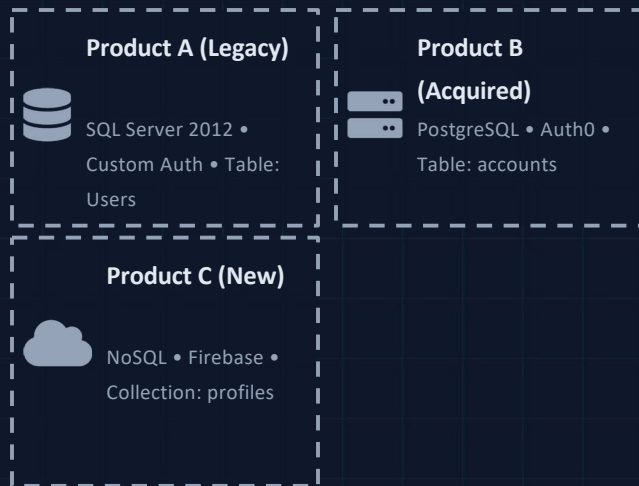
The Challenge

Identity Fragmentation in Growing SaaS Companies

Identity Chaos

Acquisitions create disconnected identity silos.

-  **Inconsistent Authentication:** Multiple login flows.
-  **Divergent User Stores:** Fragmented identity data.
-  **Operational Nightmare:** No centralized access policy.
-  **Session Friction:** Repeated logins (no SSO).



Business Impact

Why Identity Unification Matters



Revenue

- ↓ Delayed Time-to-Value
- 👤 High Churn Risk
- 🚫 Lost Cross-Sell Opportunities



Security

- ⚠️ Inconsistent Policies
- 🏠 Orphaned Accounts
- 📄 Compliance Failures



Efficiency

- 🔧 Redundant Infrastructure
- 🗣️ High Support Volume
- 🔗 Integration Complexity

75%

Faster Integration

40%

Lower Ops Costs

90%

Fewer Orphaned Accounts

60%

Fewer Support Tickets

Core Architecture

Unified Identity Platform Layers



The Central Identity Store

PostgreSQL Implementation



Why PostgreSQL?



Existing Expertise: Leveraged in-house knowledge from legacy products.



Zero Additional Investment: No new operational overhead or vendor support costs.



ACID & JSONB: Relational integrity combined with flexible user attributes.

"The primary driver was operational efficiency—using what we already knew to build something new."



Core Schema Components



Users: Centralized storage of user credentials and profiles.



Roles: Flexible RBAC with support for hierarchical roles.



Permissions: Fine-grained access control across multiple products.

Token-Time Claim Enrichment

Dynamic Authorization at Scale

1 Authentication Request

User logs in via OIDC Service. Credentials validated against core identity store.

2 Context Injection (SQL)

Stored procedure executes optimized join operations to fetch roles, tenants, and entitlements.

3 Token Generation

Contextual data is embedded into the JWT as custom claims before signing.



Zero Downstream Latency: Services validate the token locally without querying the database.

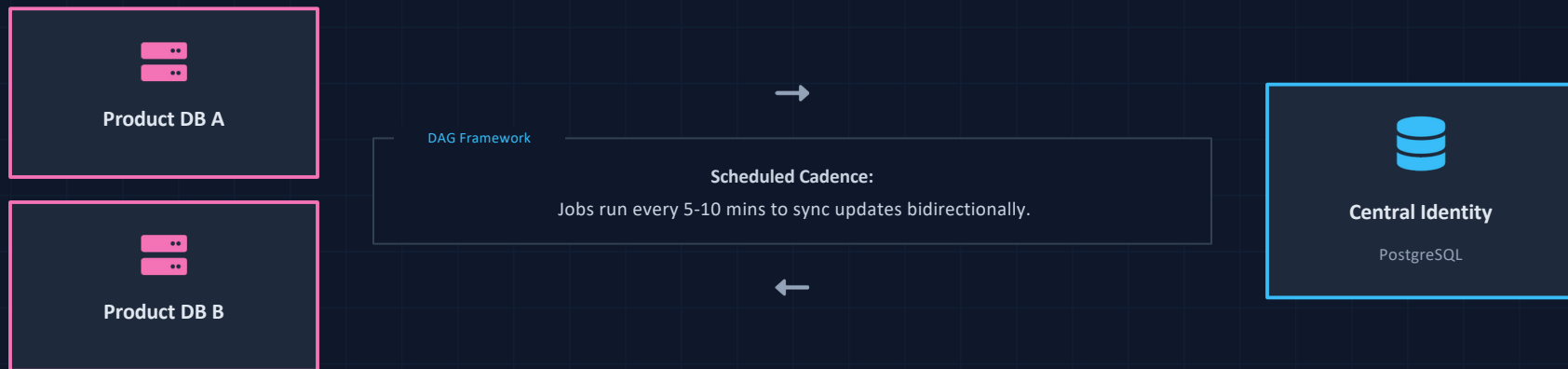
Decoded JWT Payload

JSON

```
{ "sub" : "user_12345" , "iss" : "https://auth.example.com" , "exp" :  
1735689600 , // Standard Claims above...  
// Enriched Context (Injected) "tenant_id": "org_9876", "roles": ["admin",  
"billing_manager"], "products": { "crm": { "plan": "enterprise" }, "analytics": {  
"access": "read_only" } }  
}
```

Integration Strategy

Bidirectional Data Replication



Migration Safety:

Bidirectional sync ensures that if a migration fails, we can switch back to the old model instantly without any data loss.

Integration Strategy

Bidirectional Sync via Workflows & APIs



Central Identity

PostgreSQL

Real-Time



API Calls: Immediate propagation of critical updates (e.g., password changes).

Scheduled



Workflow Cadence: Periodic sync to ensure eventual consistency across all nodes.



Product SaaS

Local Identity Store



Resilience: Background jobs automatically pick up and retry any failed API requests, ensuring no update is lost.

Implementation Detail

Enterprise-Ready Schema Design

users

id	UUID (PK)
email	VARCHAR(255)
password_hash	TEXT
email_history	JSONB

Tracks email changes and historical identifiers to prevent account fragmentation.

customers (orgs)

id	UUID (PK)
name	VARCHAR(255)
org_identifier	VARCHAR(100)

Centralized mapping of enterprise customers and their unique organizational identifiers.

permissions

user_id	UUID (FK)
org_id	UUID (FK)
product_id	UUID (FK)
action	VARCHAR(50)

product_identifiers

user_id	UUID (FK)
product_id	UUID (FK)
external_id	VARCHAR(255)

Maps central user IDs to legacy product-specific identifiers.

Performance Optimization

PostgreSQL Tuning for Identity Lookups



Indexing Strategy

B-tree indexes for primary keys and emails. GIN indexes for efficient searching within JSONB user attributes.

```
CREATE INDEX ... USING GIN
```



Caching Layer

Offload session lookups and frequently accessed roles to Redis to minimize database hits for every request.

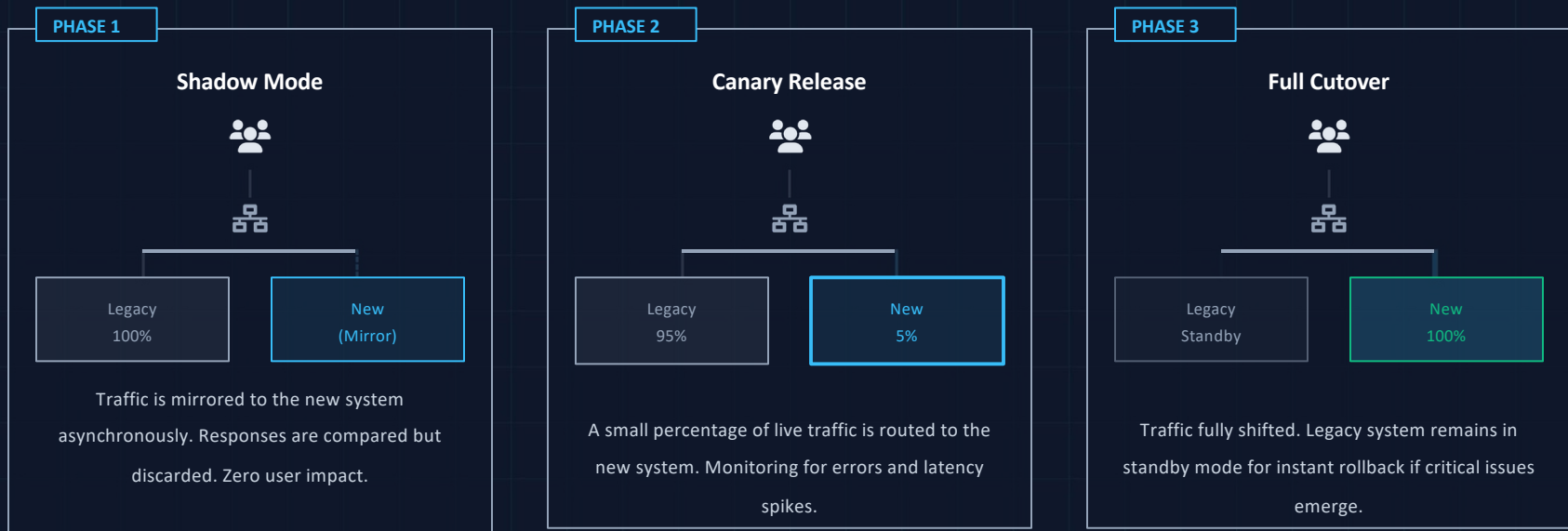
Redis Session Cache

Result:

Sub-millisecond lookup times for authentication and authorization checks, even under high concurrent load.

Migration Strategy

Near-Zero Downtime Transition



Risk Mitigation

Shadow mode allows testing with real production data volume without risking user experience.

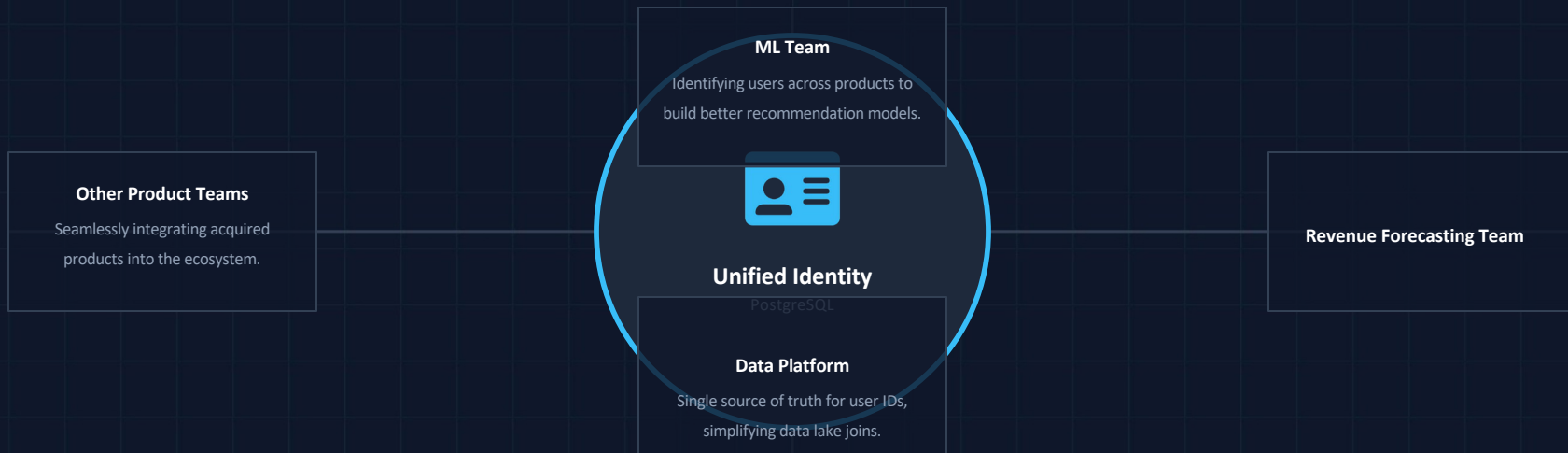


Instant Rollback

Dual-write or event replication keeps the legacy database in sync during the transition window.

The "Golden Record"

Beyond Authentication: Strategic Value



Strategic Impact:

What started as an auth project became the foundation for the company's entire data and machine learning strategy.

Vendor Strategy

Build vs. Buy & OIDC Flexibility



The In-House Build



Core Identity: Built everything in-house except the OIDC part.



Data Control: Kept critical user and org data under our own control.



The OIDC Advantage



Federated Identity: Used OIDC for external federation (Google, Azure AD).



Low Switching Cost: Easy to switch OIDC vendors with minimal effort.



Standardized: Leveraged industry standards for maximum compatibility.

"Building the core in-house while using OIDC for federation gave us the best of both worlds: control and flexibility."

Monitoring & Alerting

API-Level Visibility & Error Tracking



Infrastructure (Baseline)



CPU Utilization



Memory Usage



Disk I/O & Storage

Necessary for health, but silent during application-level failures.



Application (Critical)



401 Unauthorized Rate



403 Forbidden Rate



API Latency (p99)

Proactive Alerting: Spikes in 401/403 errors immediately alert engineers to investigate potential auth issues or bad actors.

Lessons Learned

Schema Evolution & Feature Flags



The Challenge



Null Pointer Exceptions: Encountered NPEs during the initial rollout due to incomplete data states.



Sync Orchestration: Data replication issues when turning on feature flags for specific customers.



The Lesson



Feature Flag Control: Use feature flags to carefully control the initiation of data replication.



Early Schema Design: Invest heavily in schema design from day one to avoid costly migrations later.

"Schema design is the foundation; get it right early to avoid operational pain during the rollout."

Vendor Coupling Risks

The High Cost of "Buy" vs. "Build"

The Vendor Trap

MAU Pricing Cliffs

SaaS identity providers charge per Monthly Active User. As you scale, costs grow exponentially, punishing your success.

Data Hostage

Exporting password hashes from a proprietary cloud vendor is often impossible or restricted.

Abstraction Strategy

Federate, Don't Embed

Use vendors only for Authentication (AuthN). Keep Authorization (AuthZ) and User Profiles in your own SQL Server.

Token Sovereignty

Issue your own internal JWTs. If you switch vendors, your internal services don't need to change a single line of code.

\$Cost at Scale

(1M Users)

Cloud Vendor

\$25,000 / mo

SQL Server

\$800 / mo

Questions?

"A unified identity platform is the backbone of a seamless multi-product SaaS experience."

Thank You!

For your time and attention.